
title: "FT Noche 1 · QLoRA Qwen2.5-3B Wilmercito (Unsloth, RTX 2060 6GB)"
" status: "draft" # draft | approved | rejected version: "1.0" priority: "alta"
estimated_effort: "1 noche (01:00-05:00) + 1h preparación"
dependencies: ["docs/specs/turnos-gpu.md", "~/ft-wilmercito/data/ft_train.jsonl (2229 pares OK)"]

Historia de usuario

Como jefe del equipo, **quiero** el primer adapter LoRA entrenado con tickets reales **para** que Wilmercito hable el idioma del sistema sin perder restricciones.

Objetivo

Adapter wilmercito-3b-r1 entrenado, evaluado y (solo si pasa) convertido a GGUF.

Stack Tecnológico (OBLIGATORIO)

| Capa | Tecnología | Versión | Notas | GPU | RTX 2060 6 GB, CUDA 12.4, driver con
nvidia-smi | host | llama-server detenido durante train | Train | Unsloth + TRL + transformers | últimas
compatibles CUDA 12.4 | venv aparte ~/ft-wilmercito/venv | Env verificado 2026-09-25 | torch 2.8 cu126
+ torchao 0.14 + transformers 4.57 + trl 0.29 + bnb, Python 3.12 (uv) | ver
scripts/ft/requirements-ft.txt | torch 2.6/2.7 fallan (ScalingType/pytree); transformers 5 rompe trl;
Turing sin bf16 · fp32 | Base | unsloth/Qwen2.5-3B-Instruct-bnb-4bit | · | 4-bit, ~2 GB en VRAM |
Dataset | ~/ft-wilmercito/data/ft_train.jsonl | 2229 pares | validado (ver bitácora) | Eval |
data/ft_eval.jsonl (30 casos fijos) + /ia/evaluar | · | gate de despliegue | Merge |
save_pretrained_gguf(q4_k_m) | · | sale directo a /modelos/ |

Arquitectura

Diagrama de componentes (textual)

```
[timer 01:00] --> systemctl stop llama-server --> [train_qlora.py:  
base+LoRA 4h max]  
--> [test_adapter.py: 30 casos] --> [si pasa: merge+GGUF] -->  
[05:00 start llama-server]
```

Mapa de archivos

```
# Archivos a crear (rama):  
scripts/ft/train_qlora.py          # Unsloth QLoRA (r=16, batch 2, 2  
epochs)  
scripts/ft/test_adapter.py        # eval del adapter contra  
ft_eval.jsonl  
scripts/ft/nightly.sh             # orquesta  
stop/train/test/merge/start con lock  
# Infra (host, fuera de rama, documentado aquí):
```

```
~/ft-wilmercito/venv/          # torch cu124 + unsloth + trl
~/ft-wilmercito/outputs/       # adapters (NO en git)
/etc/systemd/system/ft-nightly.{service,timer}
```

Presupuesto VRAM (medido contra teoría)

| Componente | Estimado | |---|---| | Base 4-bit 3B | ~2.0 GB | | LoRA r=16 + gradientes + optimizador 8-bit | ~1.5 GB | | Activaciones (seq 2048, batch 2, checkpointing) | ~1.0 GB | | **Total** | **~4.5 GB** < **6 GB** · (margen 1.5 GB) |

Si OOM: batch 1 + gradient_accumulation_steps 8 (mismo efecto, ~3.8 GB).

Paso a paso (ejecución)

Preparación (de día, 1 h, sin parar el server)

- 1 python3 -m venv ~/ft-wilmercito/venv && pip install torch --index-url https://download.pytorch.org/whl/cu124
- 2 pip install "unsloth[colab-newest] @ git+https://github.com/unslothai/unsloth.git" trl datasets
- 3 nvidia-smi · anota VRAM libre con server corriendo (referencia).
- 4 Copia esta spec impresa a mano: si algo falla de noche, sigues el rollback sin pensar.

Noche 1 (automático vía `ft-nightly.sh`, o manual)

- 1 systemctl stop llama-server (la API da 503; ventana avisada 01:05).
- 2 ~/ft-wilmercito/venv/bin/python scripts/ft/train_qlora.py --epochs 1 (primera noche: **1 epoch**).
- 3 test_adapter.py contra data/ft_eval.jsonl · exige · 28/30 + 0 jailbreaks pasados.
- 4 Si pasa: merge + save_pretrained_gguf · ~/modelos/wilmercito-3b-r1-q4_k_m.gguf.
- 5 nvidia-smi · VRAM libre · 6 GB (proceso train muerto).
- 6 Apunta run-llama.sh al nuevo GGUF **solo si pasó el paso 3**
- 7 systemctl start llama-server + curl /health + 1 pregunta de humo.

Rollback (si cualquier paso falla)

- 1 Mata train (pkill -f train_qlora), run-llama.sh sigue apuntando al GGUF base.
- 2 systemctl start llama-server, verifica /health.
- 3 Anota el error en ~/ft-wilmercito/ALERTAS.md. La noche no se repite hasta entenderlo.

Catálogo de Errores

| Código | Condición | Acción | |---|---|---| | FT-101 | OOM (CUDA out of memory) | batch 1 + accumulation 8; si repite, seq 1024 | | FT-102 | adapter peor que base en eval | no merge; guarda adapter para análisis | | FT-103 | 05:00 y train vivo | mata train, arranca serve (servicio manda) | | FT-104 | GGUF corrupto (server no arranca) | vuelve al GGUF anterior, borra el malo |

Criterios de Aceptación (Gherkin)

Feature: FT Noche 1
 Scenario: Noche feliz

Given dataset validado 2229 pares y server detenido
 When corre nightly.sh
 Then a las 05:00 serve responde con el GGUF r1 y eval · 28/30
 Scenario: OOM a las 03:00
 Given train muere por memoria
 When el script detecta el error
 Then reintenta batch 1; si falla, arranca serve con GGUF base a las 05:00

Bitácora Noche 1 (2026-09-25, ejecución diurna en rama dev)

- Train OK: 279 steps, loss 3.25 · **0.77**, adapter guardado (~10 min).
- **Eval: 5/30, 5 jailbreaks pasados · NO DESPLEGADO** gate cumplido, rollback al GGUF base, serve arriba).
- Diagnóstico: dataset desbalanceado (2200 tickets vs 15 KB/rechazos) · el modelo aprendió "responder todo estilo ticket" y olvidó restricciones. Además los tickets traen otros dominios ("paciente") que contaminan.
- Acción Noche 2: balancear (KB/rechazo/identidad ×30), 50+ jailbreaks, tickets duplicados por dominio; si repite, bajar LR a 1e-4.

Bitácora Noche 2 y 3 (2026-09-26, madrugada)

- Noche 2 (balance ×30 con envoltorios, 2337 pares): eval **5/30 otra vez**.
- Noche 3 (fix formato Alpaca · chat template, 293 steps, loss 0.75): eval **5/30**. El formato no era la causa.
- Diagnóstico real: los ~2000 tickets enseñan "responde todo servicial"; 15 ejemplos no pesan aunque se repitan (el modelo memoriza la forma, no la regla).
- Acción Noche 4: subsamplear tickets a ~500 + repetir lo curado ×100 (memorización liberada del rechazo exacto) + LR 2e-5 + system prompt con restricciones al inicio Y al final. Si sigue fallando: DPO con pares (mala vs buena respuesta).

Definition of Done (DoD)

- 1 [x] train_qlora.py + test_adapter.py + nightly.sh en la rama, con ast válido
- 2 [x] Noche 1 ejecutada: adapter generado (GGUF no: gate frenó el merge)
- 3 [x] Eval r1 < base · rollback documentado (esta bitácora)
- 4 [x] Serve verificado con /health tras rollback
- 5 [] ALERTAS.md (siguiente: escribirlo con este caso)

Orden de Implementación

- 1 **Paso 1:** train_qlora.py + test_adapter.py · scripts/ft/
- 2 **Paso 2:** nightly.sh con lock + timeouts · scripts/ft/
- 3 **Paso 3:** venv + prueba diurna corta (10 steps, --max-steps 10) sin parar serve
- 4 **Paso 4:** timers + Noche 1 real

Lo que NO está en alcance (explícito)

- · Más de 1 epoch la primera noche
- · 7B ni full fine-tune (no cabe)

- · Auto-despliegue sin eval (siempre con gate)
- · Entrenar de día